

Design and Implementation of an ESP32-Based IoT Smart Energy Meter

Prototype Research Paper

Abstract - This paper presents the design and preliminary implementation of an IoT-enabled single-phase energy-monitoring prototype based on the ESP32 microcontroller. The system acquires AC voltage and current using ZMPT101B and ACS712 sensing modules, respectively. The ESP32 calculates an approximate power value, displays parameters on a 16x2 I2C LCD, retains a cumulative value in EEPROM, and publishes live readings to a Blynk mobile dashboard through Wi-Fi. Prototype evidence confirms end-to-end acquisition and display of voltage, current, power, and unit values. The present work is limited to IoT energy monitoring; calibration and time-correct energy integration remain required before the measurements can be considered reference-grade energy data.

Keywords: ESP32, IoT, smart energy meter, ZMPT101B, ACS712, Blynk, voltage sensing, current sensing, energy monitoring.

1. Introduction

Conventional electricity meters generally provide consumption information at long billing intervals and offer limited visibility of instantaneous electrical behaviour. Low-cost IoT monitoring can make voltage, current and power information available at the point of use as well as remotely through a mobile interface. Such visibility is useful for observing load behaviour, checking device operation, and providing a foundation for later data-driven analysis.

The objective of this work is to design and validate an ESP32-based prototype that measures basic electrical parameters, presents them locally and remotely, and retains a cumulative unit value. This paper documents only the implemented monitoring prototype. It does not claim calibrated billing-grade energy measurement.

2. System Objectives

Objective	Implementation status
Measure AC voltage	Implemented using ZMPT101B and ESP32 ADC input.
Measure AC current	Implemented using ACS712 and ESP32 ADC input.
Calculate/display power	Approximate power calculated as voltage multiplied by current.
Provide local and remote monitoring	Implemented through LCD, serial monitor and Blynk dashboard.
Preserve accumulated value	Implemented through EEPROM read/write operations.

3. Hardware Architecture

Figure 1 shows the assembled breadboard prototype. The principal hardware elements are the ESP32 development board, an ACS712 current sensor, a ZMPT101B voltage-sensor module, a 16x2 I2C LCD, a connected test load, and a 5 V regulated/USB supply for the low-voltage electronics. The sensing arrangement supplies analog measurement signals to the ESP32; it does not use the mains input as a direct power supply for the controller.

Module	Function in the proposed system	Interface / note
ESP32	Central controller; reads analog signals, runs measurement logic and provides Wi-Fi connectivity.	GPIO 34 for ACS712; GPIO 35 for ZMPT101B in current firmware.
ZMPT101B	Isolated AC voltage sensor. It transforms the voltage waveform into a low-level measurement signal for an ADC interface.	A sensing element, not a power supply or a load-power converter.
ACS712	Hall-effect sensor that produces an analog output proportional to the current through its conductor path.	Used to estimate load current; requires calibration.
16x2 LCD	Local visual display of voltage, current, power and unit value.	I2C address 0x27 in current firmware.
Blynk	Cloud/mobile dashboard for displaying live values.	Virtual pins V0 to V3.
EEPROM	Non-volatile storage for cumulative unit variable.	512-byte allocation; value stored at address 0.

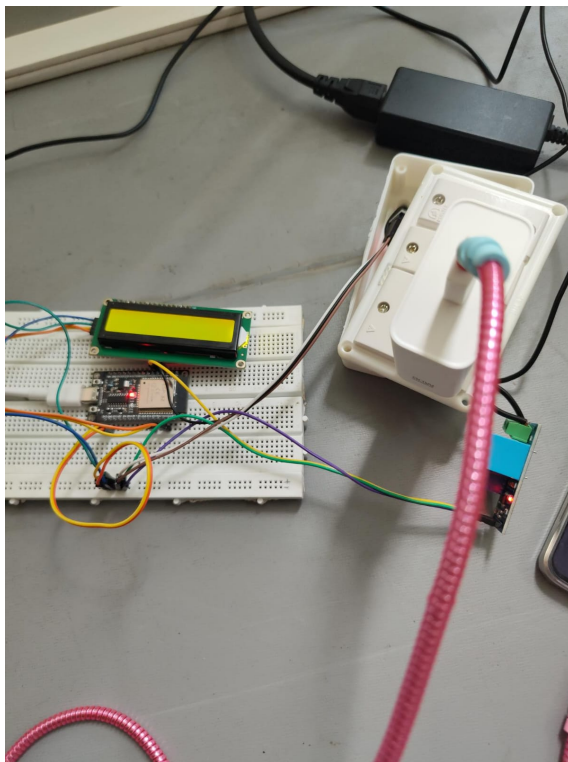


Figure 1. Breadboard implementation with LCD, ESP32, sensors and test load.

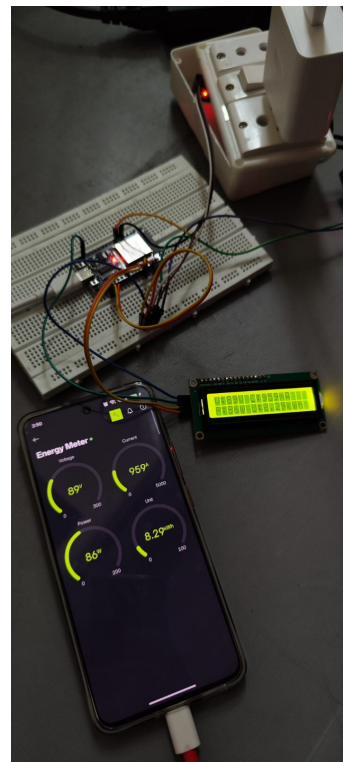


Figure 2. Blynk dashboard displaying live prototype values.

4. Operating Principle

The system uses two independent measurements to obtain electrical parameters. The ZMPT101B module senses the AC supply voltage. It includes a small precision voltage transformer and conditioning circuitry that generate an isolated, low-amplitude analog waveform. The ESP32 samples this waveform and the library converts it to an RMS-voltage estimate. The module is used for measurement; it neither powers the ESP32 nor supplies usable output power to the load.

The ACS712 measures current using the Hall effect. Current flowing through the sensor conductor produces a magnetic field; the integrated Hall sensor converts this magnetic field into an analog voltage. The firmware averages multiple AC-current samples and compensates for sensor noise. The current library is configured for GPIO 34, 3.3 V reference, 12-bit ADC resolution and a 125 A setting in the supplied sketch; the actual sensor variant and the library setting must match the physical hardware before final calibration.

The firmware obtains approximate electrical power using Equation (1).

$$P_{\text{approx}} = V_{\text{RMS}} \times I_{\text{RMS}} \quad (1)$$

Equation (1) yields apparent power in volt-amperes when the power factor is unknown. It approximates real power in watts only when the power factor is close to one. Therefore, the prototype power display should be described as an approximate value until power factor is measured or an appropriate energy-metering method is introduced.

The LCD provides immediate local visibility. In parallel, the ESP32 sends voltage, current, power and unit values to Blynk virtual pins V0, V1, V2 and V3. EEPROM is used to preserve the cumulative value through a restart, subject to a dedicated persistence test.

5. Firmware Implementation

The program initializes serial communication, EEPROM, ACS712 calibration, ZMPT101B sensitivity, LCD display, Wi-Fi connectivity and Blynk authentication. The main loop runs Blynk, calculates averaged current, measures RMS voltage, updates display/serial text, saves the unit variable to EEPROM and transmits the four values to Blynk.

```
ACS712 ACS(34, 3.3, 4095, 125);
ZMPT101B voltageSensor(35, 50.0);

float voltage = voltageSensor.getRmsVoltage();
float watt = voltage * (mA / 1000.0);

Blynk.virtualWrite(V0, volt);
Blynk.virtualWrite(V1, current);
Blynk.virtualWrite(V2, power);
Blynk.virtualWrite(V3, unit);
```

6. Prototype Results

The supplied serial-monitor evidence confirms recurrent acquisition and display of voltage, current, power and accumulated-unit values. Four sequential samples visible in the monitor are reproduced in Table 2. The Blynk display presents the same parameter categories, confirming that the Wi-Fi communication and dashboard path are active.

Table 2. Preliminary values observed in the serial monitor.

Sample	Voltage (V)	Current (mA)	Displayed power (W)	Displayed unit (kWh)
1	89	977	87	8.2633
2	89	959	86	8.2873
3	91	975	88	8.3119
4	90	976	88	8.3365

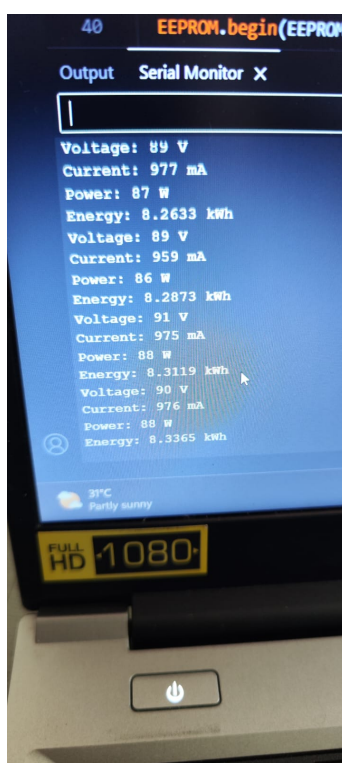


Figure 3. Serial-monitor output generated by the prototype firmware.

The displayed voltage lies between 89 V and 91 V; current lies between 959 mA and 977 mA. The approximate power reported by the firmware is 86-88 W. These results demonstrate a stable functioning data path, but they are preliminary values and require calibration against a reference meter.

7. Calibration, Limitations and Safety

The present prototype has three principal limitations. First, sensor outputs require calibration using a trusted multimeter or reference energy meter. The ZMPT101B sensitivity adjustment and ACS712 offset/current calibration should be verified under several known loads. Second, the approximation $P = V \times I$ does not account for power factor. Third, the current cumulative-energy code adds watt/3600 once per 500 ms loop without using the actual elapsed time. Consequently, the present displayed unit value must not be represented as calibrated kWh data.

The correct general energy-integration relation is:

$$\text{Energy (kWh)} = \text{Power (W)} \times \text{elapsed time (s)} / 3,600,000$$

In the final firmware, elapsed time should be obtained from `millis()`, rather than assumed from a fixed delay. The prototype should also be transferred from an exposed breadboard arrangement to an insulated enclosure with appropriate fuse protection, terminal covers, strain relief and qualified supervision for mains-related testing.

8. Discussion

The implementation demonstrates the essential components of an IoT energy-monitoring system: sensing, embedded processing, local display, retained state and remote dashboard access. The use of ESP32 is appropriate because it combines Wi-Fi connectivity with analog data acquisition in a low-cost development platform. ZMPT101B is suitable for isolated measurement of single-phase AC voltage, while ACS712 offers a simple Hall-effect approach for current sensing. Blynk enables rapid proof of remote monitoring without requiring a custom application at the prototype stage.

The most important technical outcome is not the uncalibrated numerical values themselves, but the successful sensor-to-dashboard data flow. The next engineering priority is to produce traceable measurements by correcting energy integration and validating sensor readings against known standards.

9. Conclusion

An ESP32-based IoT smart energy-meter prototype was designed and implemented. The prototype acquires voltage and current signals, presents approximate power and unit values on an LCD and serial monitor, stores a cumulative variable in EEPROM, and publishes live parameters to a Blynk mobile dashboard. The evidence demonstrates that the sensing, display and communication paths are operational. Accurate energy accounting requires time-based integration, calibration and safe enclosed construction; these steps are required before the system is used for reliable consumption measurement.

References

- [1] Allegro MicroSystems, *ACS712: Fully Integrated, Hall-Effect-Based Linear Current Sensor IC*, product documentation. Available: <https://www.allegromicro.com/en/products/sense/current-sensor-ics/integrated-current-sensors/acs712>
- [2] Qingxian Zeming Langxi Electronic Devices, *ZMPT101B Voltage Transformer Specifications*. Electrical characteristics include 50-60 Hz operation and measurement applications.
- [3] Blynk Inc., *Blynk Documentation*. Available: <https://docs.blynk.io/>
- [4] Espressif Systems, *ESP32 Series Datasheet*. Available: <https://www.espressif.com/en/support/documents/technical-documents>